

File Path Traversal

Validation of File Extension with Null Byte Bypass — Nivel Apprentice

Alumno	Manuel Eduardo Hernández Gaytán
Matrícula	181850
Materia	CNO IV – Seguridad Informática
Laboratorio	Actividad 12 · File Path Traversal – Validation of File Extension with Null Byte Bypass
Nivel	Apprentice
Plataforma	PortSwigger Web Security Academy
Docente	Mtro. Servando López Contreras
Fecha	26 de septiembre de 2026

Índice

1. Marco Teórico y Fundamentación Técnica
2. Identificación del Laboratorio
3. Objetivo del Laboratorio
4. Explicación Técnica de la Vulnerabilidad
5. Procedimiento Paso a Paso
6. Capturas de Pantalla Obligatorias
7. Explicación Técnica del Payload
8. Mitigación Recomendada
9. Conclusión

1. Marco Teórico y Fundamentación Técnica

El Path Traversal es una vulnerabilidad de seguridad web que permite a un atacante acceder a archivos arbitrarios del servidor mediante la manipulación de parámetros que construyen rutas del sistema de archivos. Una medida de mitigación habitual, distinta de bloquear directamente las secuencias `../`, consiste en validar que el nombre de archivo proporcionado por el usuario termine con una extensión permitida (por ejemplo, `.png` o `.jpg`), asumiendo que así solo se podrán servir archivos de imagen legítimos. Esta variante del laboratorio explota una inconsistencia entre el componente que realiza esa validación de extensión y el componente de bajo nivel que finalmente abre el archivo en el sistema operativo.

El bypass se apoya en el carácter nulo o "null byte" (representado como `%00` en su forma codificada para URL, y como el byte `0x00` una vez decodificado). Muchas funciones de manejo de cadenas basadas en C, y por extensión numerosas bibliotecas y tiempos de ejecución contruidos sobre ellas, interpretan el byte nulo como el terminador de la cadena. Si la aplicación valida la cadena completa (que incluye la extensión final) pero el sistema de archivos subyacente trunca la cadena en el primer byte nulo que encuentra, es posible construir una entrada que "aparente" tener la extensión correcta para la validación, mientras que en la práctica el sistema abre un archivo completamente distinto, ubicado en la ruta que precede al byte nulo.

Estándar	Identificador	Descripción
CWE	CWE-22 / CWE-626	Improper Limitation of a Pathname to a Restricted Directory / Null Byte Interaction Error
OWASP Top Ten	A01:2021	Broken Access Control — validación de entradas insuficiente
CVSS v3.x	Base Score: 7.5 (High)	Acceso no autorizado a archivos del sistema sin autenticación
CVSS Vector	AV:N/AC:L/PR:N/UI:N	Explotable remotamente, sin privilegios, sin interacción del usuario

La raíz de la vulnerabilidad reside en que dos componentes distintos de la aplicación interpretan la misma cadena de manera diferente: el componente de validación la trata como una secuencia completa de caracteres hasta su extensión final, mientras que el componente de apertura de archivos la trata como una cadena terminada en el primer byte nulo. Cualquier control de seguridad que no comparta exactamente la misma semántica de interpretación de cadenas que el componente que realiza la operación sensible puede ser eludido de esta manera.

2. Identificación del Laboratorio

Campo	Detalle
Nombre del laboratorio	File Path Traversal — Validation of File Extension with Null Byte Bypass
Nivel	Apprentice
Tipo de explotación	Path Traversal / Null Byte Injection Bypass
Plataforma	PortSwigger Web Security Academy
Posición en la ruta	Actividad 12 — Road to Hall of Fame
Herramienta principal	Burp Suite Community Edition v2026.8
Protocolo explotado	HTTP/2
Vector de ataque	Parámetro filename del endpoint /image
Técnica de bypass	Byte nulo (%00) que trunca la ruta real antes de la extensión validada (.png)
URL del laboratorio	https://0a5a000e0359d9ff80ef3fe90029009f.web-security-academy.net/

3. Objetivo del Laboratorio

Objetivo general:

Demostrar de forma práctica y documentada la explotación de una variante de Path Traversal en la que la aplicación valida que el parámetro filename termine con una extensión de imagen permitida (.png), pero delega la apertura del archivo a una función que interpreta el byte nulo como fin de cadena. Mediante la construcción de un payload que combina una secuencia de traversal con un byte nulo seguido de una extensión válida (../../../../etc/passwd%00.png), se recuperará el contenido del archivo /etc/passwd del servidor objetivo.

Objetivos específicos

- Identificar la petición HTTP vulnerable con el parámetro filename (GET /image?filename=10.jpg) en el historial de Burp Suite.
- Interceptar la petición y enviarla al módulo Repeater para su modificación controlada.
- Construir el payload ../../../../../../etc/passwd%00.png, que satisface la validación de extensión sin dejar de apuntar a /etc/passwd.
- Verificar que el servidor responde con HTTP/2 200 OK y devuelve el contenido de /etc/passwd.
- Obtener la confirmación 'Lab Solved' de la plataforma PortSwigger.

4. Explicación Técnica de la Vulnerabilidad

La aplicación del laboratorio es una tienda en línea que sirve imágenes de producto a través del siguiente endpoint:

```
GET /image?filename=10.jpg HTTP/2
Host: <lab-id>.web-security-academy.net
```

La aplicación implementa un control de seguridad que, en lugar de buscar secuencias `../`, valida que el valor de `filename` termine con una extensión de archivo de imagen permitida (por ejemplo `.png` o `.jpg`). La idea de diseño es que, si solo se pueden solicitar archivos que terminen en una extensión de imagen, no debería ser posible acceder a archivos de configuración o de sistema como `/etc/passwd`.

Mecanismo del bypass: el byte nulo. El componente de validación de extensión recorre la cadena completa y comprueba su terminación, por lo que un valor como `../../etc/passwd%00.png` supera la validación sin problema: la cadena, en su totalidad, termina en `".png"`. Sin embargo, la función de bajo nivel que finalmente abre el archivo en el sistema operativo interpreta el byte `0x00` (resultante de decodificar `%00`) como el terminador de la cadena, e ignora todo lo que aparece después. En la práctica, el sistema operativo abre el archivo correspondiente a `../../etc/passwd`, sin llegar siquiera a considerar el sufijo `%00.png`.

Valor del parámetro	Validación de extensión	Ruta que realmente se abre
10.jpg	Termina en extensión válida — pasa	/var/www/images/10.jpg
../../etc/passwd	No termina en extensión válida — bloqueado	No aplica — la petición se rechaza
../../etc/passwd%00.png	Termina en '.png' — pasa (bypass)	/etc/passwd — la cadena se trunca en el byte nulo

5. Procedimiento Paso a Paso

Paso 1: Acceso al laboratorio y configuración de Burp Suite

Se accedió al laboratorio "File path traversal, validation of file extension with null byte bypass" en PortSwigger Web Security Academy, disponible en <https://0a5a000e0359d9ff80ef3fe90029009f.web-security-academy.net/>. Se abrió Burp Suite Community Edition con la configuración predeterminada, con el proxy local activo en `127.0.0.1:8080`.

Paso 2: Carga de la página e interceptación del tráfico

Desde la pestaña Proxy se abrió el navegador integrado de Burp Suite ('Open Browser'). Se activó 'Intercept is on' y se navegó a la tienda en línea ('We Like To Shop'), quedando disponibles en el historial de Burp Suite las peticiones de carga de imágenes de producto.

Paso 3: Identificación del vector de ataque

En la pestaña Proxy se identificó la petición `GET /image?filename=10.jpg`, correspondiente a la carga de una imagen de producto. Se hizo clic derecho sobre ella y se seleccionó 'Send to Repeater' (Ctrl+R).

Paso 4: Envío de la petición original al Repeater

La petición se transfirió al módulo Repeater de Burp Suite, donde se visualizó la petición original antes de cualquier modificación:

```
GET /image?filename=10.jpg HTTP/2
Host: 0a5a000e0359d9ff80ef3fe90029009f.web-security-academy.net
```

Paso 5: Construcción y envío del payload de bypass

En el Repeater se modificó el valor del parámetro filename, sustituyendo el nombre de la imagen por una secuencia de traversal seguida de un byte nulo (%00) y una extensión de imagen válida, de modo que la validación de extensión se supere sin impedir el acceso a /etc/passwd. Se presionó el botón 'Send':

```
# Petición modificada con payload de bypass:
GET /image?filename=../../../../etc/passwd%00.png HTTP/2
```

Paso 6: Análisis de la respuesta

El panel de respuesta del Repeater mostró el código HTTP/2 200 OK, con encabezado Content-Type: image/png (reflejo de que la validación de extensión fue superada), pero el cuerpo de la respuesta contenía el archivo completo /etc/passwd del sistema operativo del servidor, incluyendo la lista de usuarios del sistema (root, daemon, peter, carlos, entre otros).

Paso 7: Confirmación — Lab Solved

Se actualizó la página en el navegador integrado y la plataforma PortSwigger mostró el banner de confirmación 'Congratulations, you solved the lab!', validando el éxito del bypass.

6. Capturas de Pantalla Obligatorias

Se presentan las seis capturas de pantalla que documentan visualmente el proceso completo de explotación, desde el estado inicial del laboratorio hasta la confirmación de su resolución.

Captura 1 — Estado inicial del laboratorio

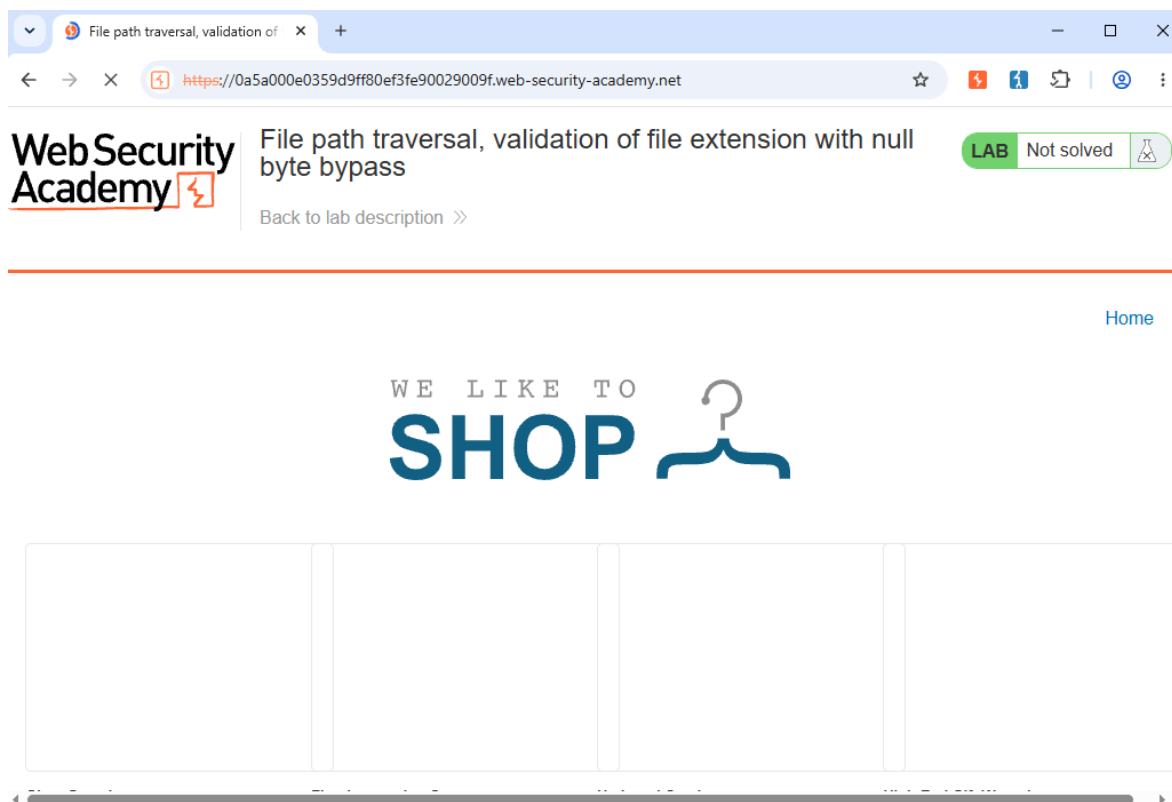


Figura 1. Estado inicial del laboratorio 'File path traversal, validation of file extension with null byte bypass' en PortSwigger Web Security Academy, marcado como 'Not solved'.

Captura 2 — Petición interceptada y enviada al Repeater

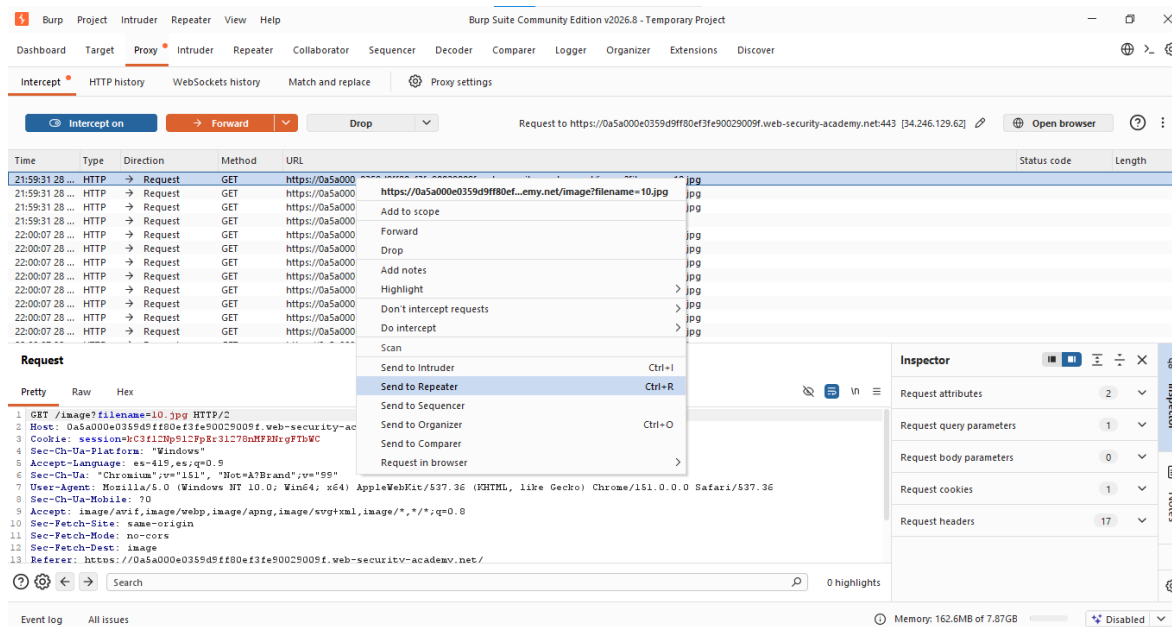


Figura 2. Historial HTTP de Burp Suite con la petición GET /image?filename=10.jpg interceptada, mostrando la opción 'Send to Repeater' seleccionada.

Captura 3 — Petición original en el Repeater

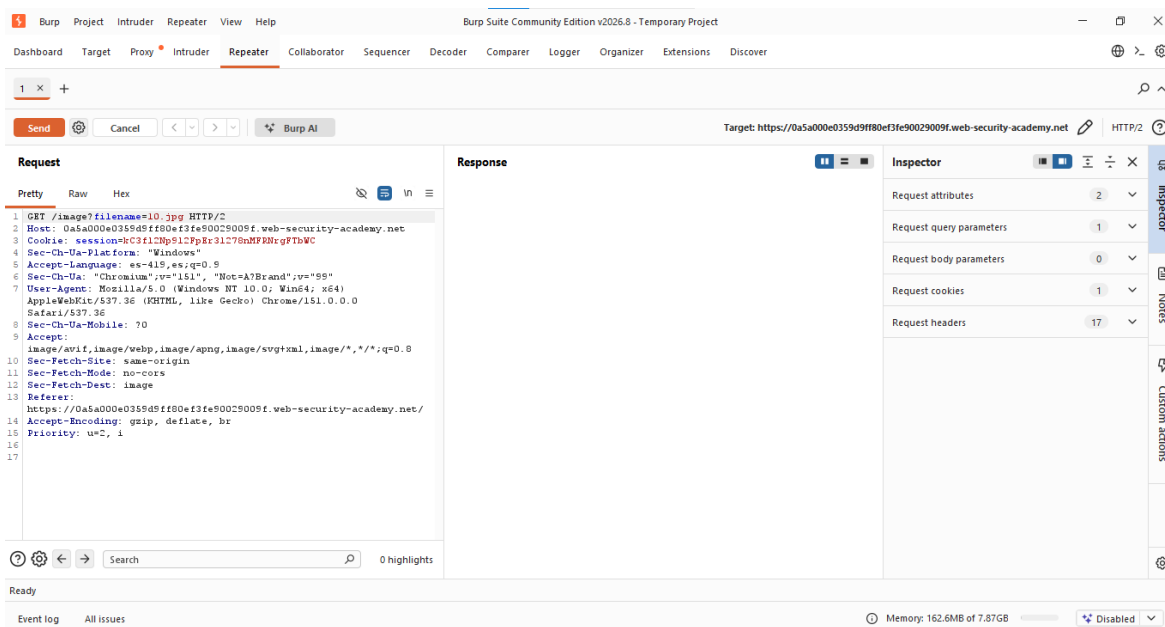


Figura 3. Petición original en el módulo Repeater de Burp Suite, lista para ser modificada.

Captura 4 — Payload de bypass enviado al servidor

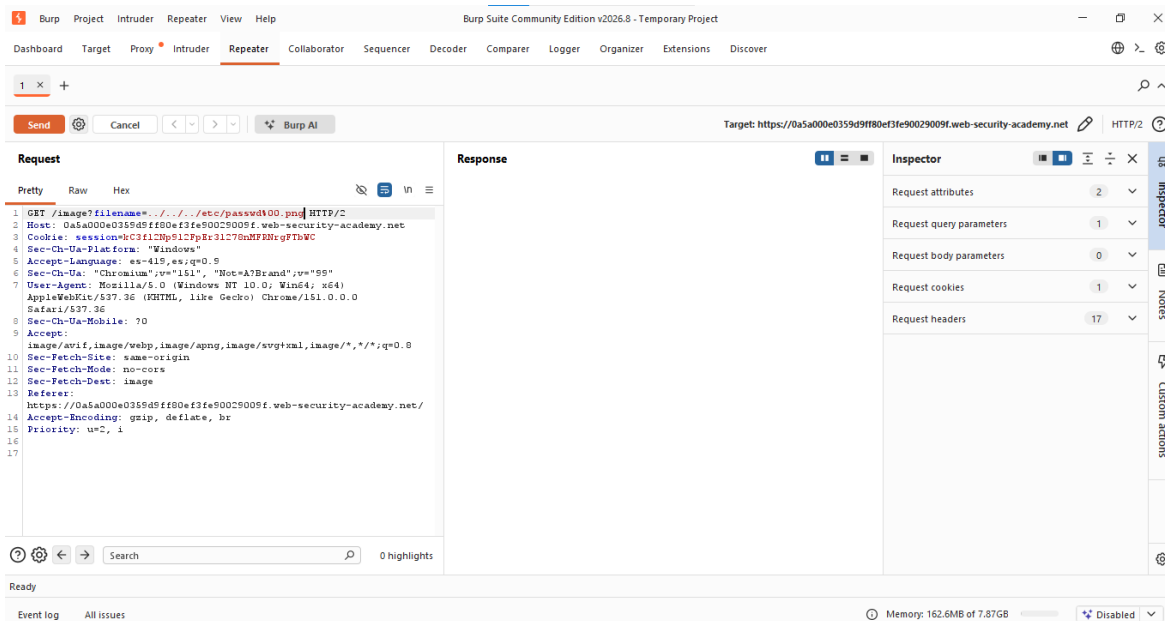


Figura 4. Petición modificada con el payload de bypass: `GET /image?filename=../../../../etc/passwd%00.png HTTP/2`.

Captura 5 — Respuesta del servidor con el contenido de /etc/passwd

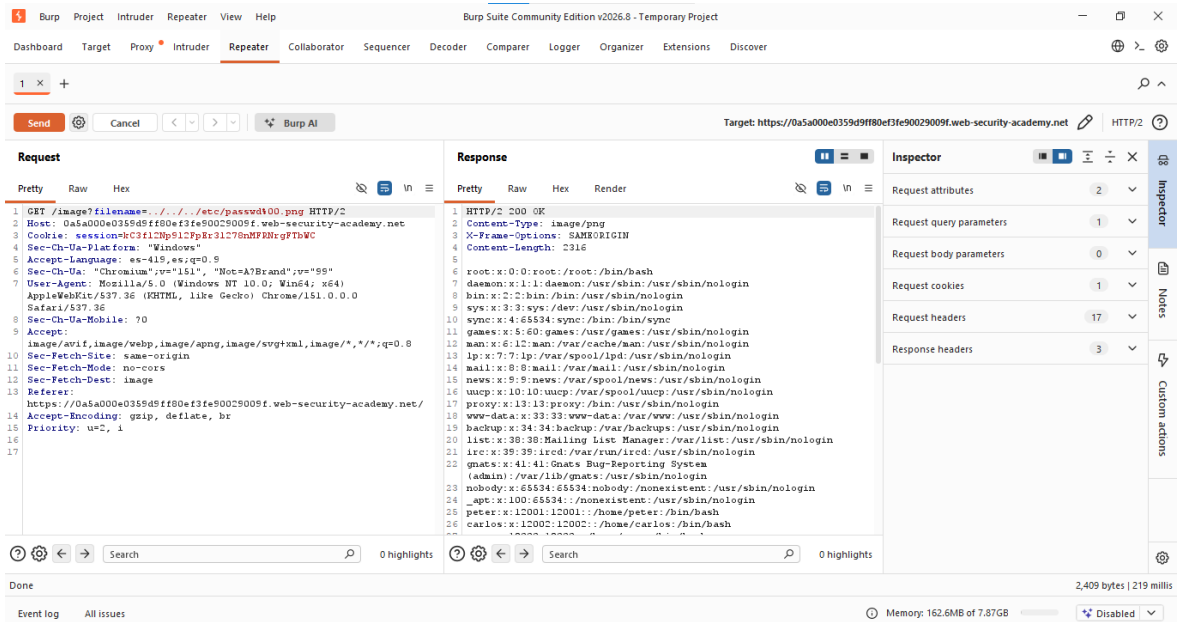


Figura 5. Respuesta HTTP/2 200 OK con Content-Type: image/png, cuyo cuerpo contiene el archivo completo `/etc/passwd`, incluyendo los registros de usuarios del sistema.

Captura 6 — Confirmación 'Lab Solved' de PortSwigger

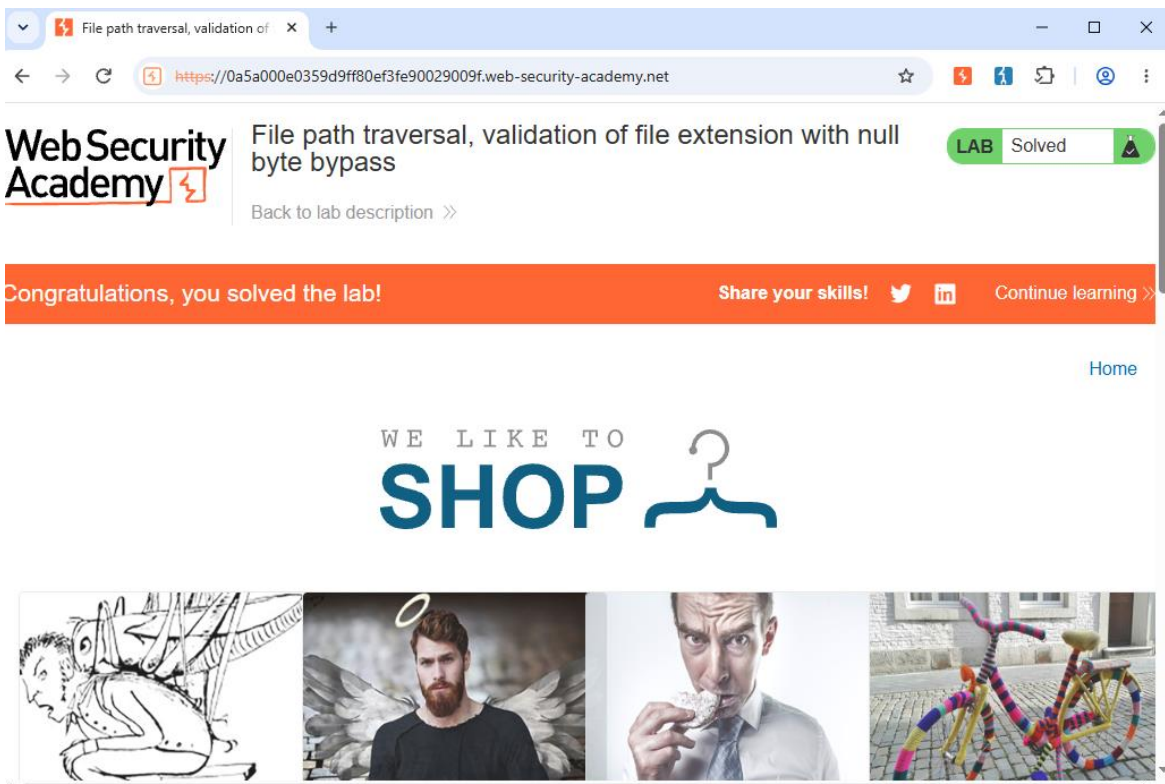


Figura 6. Banner de confirmación 'Congratulations, you solved the lab!' — laboratorio resuelto exitosamente.

7. Explicación Técnica del Payload

```
../../../../etc/passwd%00.png
```

El payload combina dos elementos que operan sobre dos componentes distintos de la aplicación:

Fragmento	Componente que lo interpreta	Función en el bypass
../../../../etc/passwd	Sistema operativo (apertura de archivo)	Ruta real que se termina abriendo, retrocediendo hasta la raíz y referenciando etc/passwd
%00	Frontera entre ambos componentes	Codificación URL del byte nulo (0x00); tras decodificar, trunca la cadena en este punto para quien la interprete como string en C
.png	Validador de extensión de la aplicación	Extensión permitida que hace que la cadena completa supere la comprobación de formato de imagen

¿Por qué dos componentes ven cadenas distintas? El lenguaje o framework de alto nivel de la aplicación (por ejemplo, en su representación interna de cadenas) puede manejar el byte nulo como un carácter más dentro de la cadena, por lo que el validador de extensión, al leer la cadena completa, encuentra correctamente que termina en ".png". Sin embargo, la llamada de sistema o la biblioteca subyacente que finalmente abre el archivo en disco proviene de una API que trata las cadenas como terminadas en el primer byte nulo (una convención heredada de C). Esa función nunca llega a ver el sufijo %00.png: para ella, la cadena termina justo antes, en ../../../../etc/passwd.

8. Mitigación Recomendada

8.1 Canonicalización y verificación de la ruta resultante (prioridad crítica)

La medida más robusta consiste en no depender de la validación de extensión como control de seguridad y, en su lugar, resolver la ruta canónica completa del archivo solicitado y verificar que permanece dentro del directorio base autorizado. Esta comprobación se realiza sobre la ruta ya resuelta por el propio sistema operativo, por lo que es inmune a discrepancias de interpretación entre componentes:

```
import os

BASE_DIR = os.path.realpath('/var/www/images')

def get_image(filename: str):
    # Rechazar explícitamente cualquier byte nulo antes de continuar
    if '\x00' in filename:
        raise ValueError('Entrada no válida')
    full_path = os.path.join(BASE_DIR, os.path.basename(filename))
    canonical = os.path.realpath(full_path)
```

```
if not canonical.startswith(BASE_DIR + os.sep):
    raise PermissionError('Acceso denegado')
return open(canonical, 'rb').read()
```

8.2 Validación por allowlisting del valor del parámetro (prioridad crítica)

Definir exactamente qué valores son aceptables para el parámetro filename — un nombre de archivo simple, sin barras, puntos de retroceso ni bytes nulos — elimina la superficie de ataque desde su origen, sin depender de que ningún componente interno interprete correctamente el final de la cadena:

```
import re

ALLOWED = re.compile(r'^[a-zA-Z0-9_-]+\.(jpg|jpeg|png|gif|webp)$')

def validate(filename: str) -> str:
    if not ALLOWED.match(filename):
        raise ValueError('Nombre de archivo no permitido')
    return filename
```

8.3 Por qué validar solo la extensión es insuficiente

Validar la extensión de un nombre de archivo asume implícitamente que todos los componentes del sistema interpretan la cadena completa de la misma manera. Esa suposición no siempre es cierta: lenguajes y bibliotecas de alto nivel suelen manejar cadenas de longitud explícita (que pueden contener bytes nulos como cualquier otro carácter), mientras que numerosas API de sistema operativo y bibliotecas heredadas de C tratan las cadenas como terminadas en el primer byte nulo. Cualquier control que no tenga en cuenta esta discrepancia puede ser eludido intercalando un byte nulo entre la parte maliciosa de la entrada y el sufijo que la hace parecer válida.

Técnica de bypass	Ejemplo	¿Lo detiene validar solo la extensión?	¿Lo detiene la canonicalización?
Traversal literal sin extensión válida	../../etc/passwd	Sí (no termina en extensión válida)	Sí
Byte nulo + extensión válida	../../etc/passwd%00.png	No	Sí
Byte nulo codificado en otras variantes	../../etc/passwd\x00.jpg	No	Sí
Ruta absoluta con extensión válida	/etc/passwd%00.png	No	Sí

8.4 Principio de mínimo privilegio y diseño seguro del endpoint

- Ejecutar el servidor web con un usuario sin privilegios que solo tenga permisos de lectura sobre el directorio de archivos estáticos.
- Aplicar controles a nivel del sistema operativo (chroot, namespaces, contenedores) para confinar el proceso del servidor web.
- Evitar exponer rutas de archivo en parámetros de URL; mapear internamente un identificador abstracto (ID numérico o UUID) al archivo real.
- Usar funciones del framework para servir archivos estáticos (`send_from_directory` en Flask, `StaticFiles` en FastAPI, `X-Accel-Redirect` en Nginx), implementadas en tiempos de ejecución modernos que no truncan cadenas en el byte nulo y aplican su propia validación de ruta nativa.

9. Conclusión

La realización de este laboratorio permitió comprender una variante de bypass de Path Traversal basada en una discrepancia de interpretación de cadenas entre dos componentes de la misma aplicación. El validador de extensión de archivo, al examinar la cadena completa, aceptaba correctamente cualquier valor que terminara en una extensión de imagen permitida. Sin embargo, la función de bajo nivel que finalmente abría el archivo truncaba la cadena en el primer byte nulo, ignorando por completo el sufijo que la había hecho parecer segura ante el validador.

El caso ilustra un principio de seguridad fundamental: un control de validación solo es confiable si opera sobre exactamente la misma representación de los datos que utilizará, más adelante, el componente que ejecuta la acción sensible. Cuando dos partes del sistema interpretan la misma cadena de bytes de forma distinta —ya sea por una decodificación adicional, por una convención heredada como la terminación en byte nulo, o por cualquier otra transformación implícita— se abre una ventana de inconsistencia que un atacante puede explotar sin necesidad de romper la validación en sí misma.

El impacto del exploit es el mismo que en otras variantes de Path Traversal: lectura no autorizada del archivo `/etc/passwd` con la lista completa de usuarios del sistema operativo del servidor, información que puede aprovecharse para escalar el ataque mediante enumeración de cuentas, ataques de fuerza bruta y, en escenarios más complejos, lectura de archivos de configuración con credenciales de acceso.

La única mitigación verdaderamente robusta frente a esta familia de vulnerabilidades es no confiar en propiedades superficiales de la cadena de entrada (como su extensión aparente) y, en su lugar, resolver la ruta canónica real que el sistema operativo utilizará y verificar el confinamiento al directorio base sobre ese resultado final.